

How providers can be declared in angular?

In Angular, providers can be declared at **three different levels** of the application. Where you declare a provider is a major architectural decision because it controls two things:

1. **The Scope:** Which components can see and use the service.
2. **The Lifecycle (Bundle Size & Memory):** When the service is created, how many instances exist, and when it gets destroyed.

Here is exactly where providers are declared in modern Angular.

1. Root Level (Application-Wide Scope)

This is the most common way to declare global services. There are two ways to achieve this:

A. The `@Injectable` Decorator (Recommended)

You declare the provider directly inside the service file itself using the `providedIn: 'root'` property.

```
TypeScript
@Injectable({
  providedIn: 'root' // <-- Automatically declared at the root level
})
export class AuthService {}
```

- **Why use it:** It makes the service **tree-shakable**. If your app never imports this service, Angular's compiler strips it out of the final JavaScript bundle entirely.

B. The `app.config.ts` File

If you are bootstrapping global configurations, third-party libraries, or `InjectionTokens` that don't use the `@Injectable` decorator, you declare them in the global application configuration array.

```
TypeScript
// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: API_URL, useValue: 'https://api.com' },
    LocalStorageService // Shorthand syntax
  ]
};
```

Behavior: Root-level providers are **global singletons**. Only one instance is created for the entire application, and it stays in memory until the user closes or refreshes the browser tab.

2. Route Level (Lazy-Loaded Scope)

When you use the Angular Router to lazy-load features, you can declare providers inside your route definition configuration using the `providers` array.

How providers can be declared in angular?

TypeScript

```
// app.routes.ts
export const routes: Routes = [
  {
    path: 'dashboard',
    loadChildren: () => import('./dashboard/dashboard.routes'),

    // Declared only for this route environment subtree
    providers: [ DashboardApiService ]
  }
];
```

Behavior: This creates a **Route-level Environment Injector**. The `DashboardApiService` is isolated to the `/dashboard` route.

- Components inside the dashboard route share the *same* single instance of this service.
- Components *outside* the dashboard route cannot see or access it.
- The service code is cleanly bundled into the lazy-loaded chunk file, keeping your initial website load incredibly fast.

3. Component Level (Local Element Scope)

You can declare a provider directly inside a component's `@Component` metadata decorator using its local `providers` array.

TypeScript

```
@Component({
  selector: 'app-video-player',
  standalone: true,
  templateUrl: './video-player.component.html',

  // Declared at the local component element level
  providers: [ PlayerStateService ]
})
export class VideoPlayerComponent {}
```

Behavior: This creates an instance bound strictly to the **lifecycle of this specific component instance**.

- Every single time you use `<app-video-player>` in your HTML, Angular spins up a **brand new, unique instance** of `PlayerStateService`.
- If you have three video players on a page, you have three separate service instances tracking state independently.
- When the component is destroyed (e.g., removed via `*ngIf`), its local service instance is automatically destroyed and cleaned from memory.

Architectural Summary Reference

How providers can be declared in angular?

Declaration Location	How to Write It	Lifetime	Shared?
Root Level	<code>@Injectable({ providedIn: 'root' })</code> or <code>app.config.ts</code>	App Lifespan	Yes (One global singleton for everyone)
Route Level	<code>providers: [...]</code> inside a <code>Route</code> object	Route Lifespan	Yes (Shared <i>only</i> among child routes/components of that route)
Component Level	<code>providers: [...]</code> inside <code>@Component</code> metadata	Component Lifespan	No (Unique to that specific HTML component instance and its children)